A Practical Guide to Prompt engineering

TOPIC -- PROMPT ENGINEERING

-- 01 --

A text-to-image prompt commonly includes a description of the subject of the art, the desired medium (such as digital painting or photography), style (such as hyperrealistic or pop-art), lighting (such as rim lighting or crepuscular rays), color, and texture. Word order also affects the output of a text-to-image prompt. Words closer to the start of a prompt may be emphasized more heavily.

-- 02 --

Tree-of-thought Tree-of-thought prompting generalizes chain-of-thought by generating multiple lines of reasoning in parallel, with the ability to backtrack or explore other paths. It can use tree search algorithms like breadth-first, depth-first, or beam.

-- 03 --

Using language models to generate prompts LLMs themselves can be used to compose prompts for LLMs. The automatic prompt engineer algorithm uses one LLM to beam search over prompts for another LLM:

-- 04 --

Automated prompt generation Recent research has explored automated prompt engineering, using optimization algorithms to generate or refine prompts without human intervention. These automated approaches aim to identify effective prompt patterns by analyzing model gradients, reinforcement feedback, or evolutionary processes, reducing the need for manual experimentation.

-- 05 --

Context engineering Context engineering is a related process that focuses on the context elements that accompany user prompts, which include system instructions, retrieved knowledge, tool definitions, conversation summaries, and task metadata. Context engineering is performed to improve reliability, provenance and token efficiency in production LLM systems. The concept emphasises operational practices such as token budgeting, provenance tags, versioning of context artifacts, observability (logging which context was supplied), and context regression tests to ensure that changes to supplied

context do not silently alter system behaviour.

-- 06 --

Using gradient descent to search for prompts In "prefix-tuning", "prompt tuning", or "soft prompting", floating-point vectors are searched directly by gradient descent to maximize the log-likelihood on outputs. An earlier result uses the same idea of gradient descent search, but is designed for masked language models like BERT, and searches only over token sequences, rather than numerical vectors. Formally, it searches for arg ■ max X ~ ∑ i log ■ P r [Y i | X ~ ∗ X i] {\displaystyle \arg \max _{\tilde {X}}\sum _{i}\log Pr[Y^{i}|{\tilde {X}}\ast X^{i}]} where X ~ {\displaystyle {\tilde {X}}} ranges over token sequences of a specified length.

-- 07 --

Retrieval-augmented generation (RAG) Retrieval-augmented generation is a technique that enables GenAI models to retrieve and incorporate new information. It modifies interactions with an LLM so that the model responds to user queries with reference to a specified set of documents, using this information to supplement information from its pre-existing training data. This allows LLMs to use domain-specific and/or updated information. RAG improves large language models by incorporating information retrieval before generating responses. Unlike traditional LLMs that rely on static training data, RAG pulls relevant text from databases, uploaded documents, or web sources. By dynamically retrieving information, RAG enables AI to generate more accurate responses and fewer AI hallucinations without frequent retraining.

-- 08 --

MIPRO (Multi-prompt Instruction Proposal Optimizer) optimizes the instructions and few-shot demonstrations of multi-stage language model programs, proposing small changes to module prompts and retaining those that improve a downstream performance metric without access to module-level labels or gradients. GEPA (Genetic-Pareto) is a reflective prompt optimizer for compound AI systems that combines language-model-based analysis of execution traces and textual feedback with a Pareto-based evolutionary search over a population of candidate systems; across four tasks, GEPA reports average gains of about 10% over reinforcement-learning-based Group Relative Policy Optimization (GRPO) and over 10% over the MIPROv2 prompt optimizer, while using up to 35 times fewer rollouts than GRPO. Open-source frameworks such as DSPy and Opik expose these and related optimizers, allowing prompt search to be expressed as part of a programmatic pipeline rather than through manual trial

and error.

-- 09 --

Limitations The process of writing and refining a prompt for an LLM or generative AI shares some parallels with an iterative engineering design process, such as by discovering reusable best practices through reproducible experimentation. But the techniques that improve performance depend heavily on the specific model being used. Such patterns are also volatile and exhibit significantly different results from seemingly insignificant prompt changes.

-- 10 --

There are two LLMs. One is the target LLM, and another is the prompting LLM. Prompting LLM is presented with example input-output pairs, and asked to generate instructions that could have caused a model following the instructions to generate the outputs, given the inputs. Each of the generated instructions is used to prompt the target LLM, followed by each of the inputs. The log-probabilities of the outputs are computed and added. This is the score of the instruction. The highest-scored instructions are given to the prompting LLM for further variations. Repeat until some stopping criteria is reached, then output the highest-scored instructions. CoT examples can be generated by LLM themselves. In "auto-CoT", a library of questions are converted to vectors by a model such as BERT. The question vectors are clustered. Questions close to the centroid of each cluster are selected, in order to have a subset of diverse questions. An LLM does zero-shot CoT on each selected question. The question and the corresponding CoT answer are added to a dataset of demonstrations. These diverse demonstrations can then added to prompts for few-shot learning.

-- 11 --

Techniques Common terms used to describe various specific prompt engineering techniques include chain-of-thought, tree-of-thought, and retrieval-augmented generation (RAG). A 2024 survey of the field identified over 50 distinct text-based prompting techniques, 40 multimodal variants, and a vocabulary of 33 terms used across prompting research, highlighting a present lack of standardised terminology for prompt engineering. Vibe coding is an AI-assisted software development method where a user prompts an LLM with a description of what they want and lets it generate or edit the code. In 2025, "vibe coding" was the Collins Dictionary word of the year.

-- 12 --

Rationale Research has found that the performance of large language models (LLMs) is highly sensitive to choices such as the ordering of examples, the quality of demonstration labels, and even small variations in phrasing. In some cases, reordering examples in a prompt produced accuracy shifts of more than 40 percent.